ZESPÓŁ SZKÓŁ ELEKTRYCZNO-MECHANICZNYCH

Pracownia testowania i dokumentowania aplikacji

Testowanie aplikacji

Autor: Adrian Leśnik

Klasa: 4P

Prowadzący: mgr inż. Kamil Wojnarowski

NOWY SĄCZ 2026r.

Spis treści

Wstęp 3

1. Testy jednostkowe 4

1.1 Implementacja programu 4

1.2 Implementacja testów 5

1.3 Git 5

Podsumowanie 6

2. Testy integracyjne 7

2.1 Implementacja programu 8

2.2 Implementacja testów 10

Wnioski do rozdziału 12

3. Testy systemowe 13

3.1 Implementacja programu 14

3.2 Implementacja testów 17

Wnioski do rozdziału 20

Wstęp

Testowanie oprogramowania polega na sprawdzaniu, czy program działa zgodnie z wymaganiami i oczekiwaniami użytkownika. Jego głównym celem jest wykrycie błędów, zanim trafią one do użytkownika końcowego.

Wyróżniamy cztery podstawowe poziomy testowania: testy jednostkowe (pojedyncze metody i klasy), integracyjne (współpraca modułów), systemowe (cały system) i akceptacyjne (zgodność z wymaganiami klienta). Testy dzielimy też na czarnoskrzynkowe, wykonywane bez znajomości kodu, oraz białoskrzynkowe, które wykorzystują wiedzę o jego budowie.

Do najważniejszych zasad testowania należą: testy pokazują obecność błędów, ale nie dowodzą ich braku, nie da się przetestować wszystkiego, a im wcześniej zaczniemy testować, tym taniej naprawimy błędy. Proces testowy obejmuje planowanie, projektowanie, wykonanie i ocenę wyników testów.

W pracy opisano testy jednostkowe, integracyjne i systemowe na przykładzie programów napisanych w języku C# i testowanych w bibliotece xUnit.

1. Testy jednostkowe

Testy jednostkowe sprawdzają najmniejsze części programu, czyli pojedyncze metody lub klasy, w oderwaniu od reszty aplikacji. Wykonuje się je w trakcie pisania kodu, zwykle zaraz po napisaniu metody, oraz po każdej jego zmianie, aby upewnić się, że nic się nie zepsuło.

Przykład. Dla metody obliczającej cenę po rabacie test jednostkowy polega na wywołaniu jej z konkretnymi danymi i sprawdzeniu wyniku: dla ceny 100 zł i rabatu 20% oczekujemy 80 zł.

xUnit to darmowa biblioteka do testów jednostkowych na platformie .NET. Test to metoda oznaczona atrybutem `[Fact]`, a wynik sprawdza się metodami klasy `Assert`, np. `Assert.Equal` lub `Assert.Throws`. Testy uruchamia się poleceniem `dotnet test` albo w Test Explorerze w Visual Studio.

1.1 Implementacja programu

Testowany program to klasa `Sortowanie` z metodą `Sort`, która przyjmuje tablicę liczb całkowitych i zwraca nową, posortowaną rosnąco tablicę. Użyto sortowania bąbelkowego.

```
using System;

namespace Sortowanie_Przez_Wstawianie
{
    public class Sortowanie
    {
        public int[] Sort(int[] array)
        {
            if (array == null)
            {
                throw new ArgumentNullException(nameof(array));
            }

            int[] result = (int[])array.Clone();
            int n = result.Length;

            for (int i = 0; i < n - 1; i++)
            {
                for (int j = 0; j < n - i - 1; j++)
                {
                    if (result[j] > result[j + 1])
                    {
                        int temp = result[j];
                        result[j] = result[j + 1];
                        result[j + 1] = temp;
                    }
                }
            }

            return result;
        }
    }
}
```

Metoda najpierw sprawdza, czy tablica nie jest `null` – jeśli jest, zgłasza wyjątek `ArgumentNullException`. Następnie tworzy kopię tablicy, żeby nie zmieniać danych wejściowych. Dwie zagnieżdżone pętle porównują sąsiednie elementy i zamieniają je

miejscami, gdy lewy jest większy od prawego. Po każdym przebiegu największy element trafia na koniec, dlatego pętla wewnętrzna jest z każdym przebiegiem krótsza. Na końcu metoda zwraca posortowaną kopię.

1.2 Implementacja testów

Testy napisano w xUnit. Klasa `SortowanieTests` zawiera cztery testy sprawdzające różne przypadki.

```
using Xunit;
using System;
using Sortowanie_Przez_Wstawianie;

namespace Sortowanie_Przez_Wstawianie_Testy
{
    public class SortowanieTests
    {
        private readonly Sortowanie _sorter = new Sortowanie();

        [Fact]
        public void Test1()
        {
            int[] input = { 4, 2, 1, 3 };
            int[] expected = { 1, 2, 3, 4 };
            Assert.Equal(expected, _sorter.Sort(input));
        }

        [Fact]
        public void Test2()
        {
            int[] input = { 1, 2, 3 };
            int[] expected = { 1, 2, 3 };
            Assert.Equal(expected, _sorter.Sort(input));
        }

        [Fact]
        public void Test3()
        {
            int[] input = Array.Empty<int>();
            int[] expected = Array.Empty<int>();
            Assert.Equal(expected, _sorter.Sort(input));
        }

        [Fact]
        public void Test4()
        {
            Assert.Throws<ArgumentNullException>(() => _sorter.Sort(null));
        }
    }
}
```

Test1 sprawdza sortowanie zwykłej tablicy, Test2 – tablicy już posortowanej, Test3 – tablicy pustej, a Test4 – czy dla wartości `null` zostanie zgłoszony wyjątek `ArgumentNullException`. Po uruchomieniu `dotnet test` wszystkie cztery testy zakończyły się wynikiem pozytywnym.

1.3 Git

Do zapisywania zmian użyto systemu Git, a repozytorium znajduje się na GitHubie. Każdy test zapisano w osobnym commicie:

- `9b2fbf8` – sprawdzenie sortowania zwykłej tablicy

- `b93d938` – sprawdzenie tablicy już posortowanej
- `0ae200d` – sprawdzenie pustej tablicy
- `3f986c5` – sprawdzenie rzucania wyjątku dla null

Podsumowanie

W pracy opisano testy jednostkowe i pokazano je na przykładzie metody sortującej tablicę w języku C#. Napisano cztery testy w xUnit: dla zwykłej tablicy, tablicy posortowanej, pustej oraz dla wartości null. Wszystkie zakończyły się powodzeniem, a przebieg prac zapisano w czterech commitach w Git.

2. Testy integracyjne

Testy integracyjne (ang. integration tests) to poziom testowania, na którym sprawdzamy, czy kilka komponentów systemu współpracuje ze sobą poprawnie. Test jednostkowy weryfikuje jedną odizolowaną metodę lub klasę (zależności są wtedy zazwyczaj zastępowane atrapami), natomiast test integracyjny łączy ze sobą prawdziwe obiekty i sprawdza, czy dane poprawnie przechodzą przez granice między nimi – np. od warstwy logiki biznesowej, przez kalkulator, aż do warstwy zapisu danych.

Same komponenty mogą być w pełni poprawne, a mimo to system może działać błędnie, jeśli np. jeden z modułów przekazuje drugiemu dane w innym formacie, w niewłaściwej kolejności albo jeśli błąd w jednym module pozostawia system w niespójnym stanie. Właśnie takie problemy wykrywają testy integracyjne.

Kiedy wykonujemy testy integracyjne?

- po zakończeniu testów jednostkowych poszczególnych modułów (zgodnie z układem poziomów testowania: jednostkowe → integracyjne → systemowe → akceptacyjne),
- gdy do systemu dołączany jest nowy moduł lub zmienia się sposób komunikacji między modułami,
- przed uruchomieniem testów systemowych, aby wcześnie wyłapać błędy na łączeniach między komponentami.

Porównanie z testami jednostkowymi

Cecha	Testy jednostkowe	Testy integracyjne
Zakres	jedna klasa / metoda	kilka współpracujących komponentów
Zależności	izolowane (atrapy, mocki)	prawdziwe obiekty
Szybkość	bardzo szybkie	wolniejsze
Co wykrywają	błędy w logice metody	błędy na styku komponentów

Przykład (opisowy)

W sklepie internetowym klient składa zamówienie. Moduł obsługi zamówień przyjmuje dane, przekazuje cenę do modułu odpowiedzialnego za wyliczenie rabatu, a otrzymany wynik zapisuje w bazie danych. Test jednostkowy sprawdziłby osobno samo wyliczanie rabatu. Test integracyjny wywoła natomiast złożenie zamówienia i sprawdzi, czy w bazie znalazła się pozycja z właściwą, już pomniejszoną ceną – a więc czy wszystkie trzy elementy (obsługa zamówienia, kalkulator i baza) działają razem tak, jak zakładamy.

Narzędzie: xUnit

Do napisania testów użyto biblioteki xUnit – popularnego frameworka testowego dla platformy .NET. xUnit służy zarówno do testów jednostkowych, jak i integracyjnych, bo to nie narzędzie, lecz zakres testu decyduje o jego rodzaju. Najważniejsze elementy:

- `[Fact]` – atrybut oznaczający metodę jako test,
- `Assert` – klasa z metodami sprawdzającymi wynik (`Equal`, `NotNull`, `Empty`, `Throws`),
- `dotnet test` – polecenie uruchamiające wszystkie testy w projekcie.

Testy pisane są według wzorca AAA: Arrange (przygotuj obiekty), Act (wykonaj akcję), Assert (sprawdź wynik).

2.1 Implementacja programu

Jako program testowany zaimplementowano prosty system wystawiania faktur o nazwie **SuperProstyFaktury** (język C#, .NET). System przyjmuje nazwę klienta, cenę netto i stawkę VAT, wylicza cenę brutto i zapisuje fakturę w bazie. Baza danych jest dla uproszczenia przechowywana w pamięci (lista obiektów), dzięki czemu testy nie wymagają żadnej zewnętrznej konfiguracji.

Struktura rozwiązania

```
SuperProstyFaktury.sln
 +-- SuperProstyFaktury/        (biblioteka klas - kod produkcyjny)
 |     +-- Invoice.cs
 |     +-- TaxCalculator.cs
 |     +-- InvoiceDatabase.cs
 |     +-- InvoiceService.cs
 +-- SuperProstyFaktury.Tests/  (projekt xUnit - testy)
       +-- InvoiceIntegrationTests.cs
```

Projekty utworzono poleceniami:

```
dotnet new sln -n SuperProstyFaktury
dotnet new classlib -n SuperProstyFaktury
dotnet new xunit -n SuperProstyFaktury.Tests
dotnet sln add SuperProstyFaktury SuperProstyFaktury.Tests
dotnet add SuperProstyFaktury.Tests reference SuperProstyFaktury
```

Opis działania

System składa się z czterech klas, z których każda ma jedną, jasno określoną odpowiedzialność:

- `Invoice` – model danych faktury: nazwa klienta (`Customer`) i cena brutto (`GrossPrice`).
- `TaxCalculator` – logika podatkowa. Metoda `AddVat` dolicza VAT do ceny netto według wzoru netto + netto × VAT / 100. Dla ujemnej stawki VAT zgłasza wyjątek `ArgumentException`.
- `InvoiceDatabase` – uproszczona baza danych. Przechowuje zapisane faktury na liście `SavedInvoices`, a metoda `Save` dodaje do niej nową fakturę.
- `InvoiceService` – warstwa łącząca wszystko w całość. Metoda `CreateInvoice` najpierw wylicza cenę brutto w kalkulatorze, a dopiero potem zapisuje fakturę w bazie. Zależności (baza i kalkulator) są przekazywane przez konstruktor.

Ważna jest kolejność operacji: ponieważ wyliczenie ceny odbywa się przed zapisem, błędne dane (ujemny VAT) przerywają proces jeszcze przed dotarciem do bazy i nie zostawiają w niej żadnych śmieci. Jest to zachowanie, które dobrze nadaje się do sprawdzenia w teście integracyjnym.

Kod programu

```
// Invoice.cs
namespace SuperProstyFaktury
{
    public class Invoice
    {
        public string Customer { get; set; } = "";
        public decimal GrossPrice { get; set; }
    }
}
```

```
// TaxCalculator.cs
using System;

namespace SuperProstyFaktury
{
    public class TaxCalculator
    {
        public decimal AddVat(decimal netPrice, decimal vatPercent)
        {
            if (vatPercent < 0)
                throw new ArgumentException("VAT nie moze byc ujemny");

            return netPrice + netPrice * vatPercent / 100;
        }
    }
}
```

```
// InvoiceDatabase.cs
using System.Collections.Generic;

namespace SuperProstyFaktury
{
    public class InvoiceDatabase
    {
        public List<Invoice> SavedInvoices { get; } = new();

        public void Save(Invoice invoice) => SavedInvoices.Add(invoice);
    }
}
```

```
// InvoiceService.cs
namespace SuperProstyFaktury
{
    public class InvoiceService
    {
        private readonly InvoiceDatabase _db;
        private readonly TaxCalculator _calculator;

        public InvoiceService(InvoiceDatabase db, TaxCalculator calculator)
        {
            _db = db;
            _calculator = calculator;
        }

        public void CreateInvoice(string customer, decimal netPrice,
                                  decimal vatPercent)
        {
            var gross = _calculator.AddVat(netPrice, vatPercent);
            _db.Save(new Invoice { Customer = customer, GrossPrice = gross });
        }
    }
}
```

2.2 Implementacja testów

Dla systemu napisano cztery testy integracyjne w xUnit. W każdym z nich tworzone są trzy prawdziwe obiekty – `InvoiceDatabase`, `TaxCalculator` i `InvoiceService` – połączone ze sobą tak jak w działającej aplikacji. Nie używamy atrap, dlatego test sprawdza cały przepływ: serwis → kalkulator → baza. Każdy test tworzy własną, nową bazę, więc testy są od siebie niezależne.

Opis testów

Test	Co sprawdza	Oczekiwany wynik
`Integration_ShouldSaveInvoiceWithStandardVat`	Faktura 1000 netto, VAT 23% przechodzi przez serwis i kalkulator i trafia do bazy.	w bazie faktura klienta „Firma ABC” z ceną brutto 1230
`Integration_ShouldSaveInvoiceWithReducedVat`	To samo dla obniżonej stawki VAT 8%.	w bazie faktura z ceną brutto 540 (500 + 8%)
`Integration_ShouldNotSaveAnything_WhenVatIsInvalid`	Ujemny VAT powoduje wyjątek w kalkulatorze, a serwis nie zapisuje nic w bazie.	zgłoszony `ArgumentException`, baza pozostaje pusta
`Integration_ShouldCorrectlyAccumulateMultipleInvoicesInDatabase`	Dwie faktury zapisane po kolei w tej samej bazie.	w bazie 2 faktury: 123 oraz 200

Kod testów

Plik `InvoiceIntegrationTests.cs` w projekcie `SuperProstyFaktury.Tests`:

```
using System;
using System.Linq;
using Xunit;
using SuperProstyFaktury;

namespace SuperProstyFaktury.Tests
{
```

```
public class InvoiceIntegrationTests
{
    [Fact]
    public void Integration_ShouldSaveInvoiceWithStandardVat()
    {
        // Arrange: laczymy wszystkie 3 komponenty w jeden system
        var db = new InvoiceDatabase();
        var calculator = new TaxCalculator();
        var service = new InvoiceService(db, calculator);

        // Act
        service.CreateInvoice("Firma ABC", 1000, 23);

        // Assert
        var saved = db.SavedInvoices.FirstOrDefault(i => i.Customer == "Firma ABC");
        Assert.NotNull(saved);
        Assert.Equal(1230m, saved.GrossPrice); // 1000 + 23%
    }

    [Fact]
    public void Integration_ShouldSaveInvoiceWithReducedVat()
    {
        // Arrange
        var db = new InvoiceDatabase();
        var calculator = new TaxCalculator();
        var service = new InvoiceService(db, calculator);

        // Act: obnizony VAT 8%
        service.CreateInvoice("Ksiegarnia", 500, 8);

        // Assert
        var saved = db.SavedInvoices.FirstOrDefault(i => i.Customer == "Ksiegarnia");
        Assert.NotNull(saved);
        Assert.Equal(540m, saved.GrossPrice); // 500 + 8%
    }

    [Fact]
    public void Integration_ShouldNotSaveAnything_WhenVatIsInvalid()
    {
        // Arrange
        var db = new InvoiceDatabase();
        var calculator = new TaxCalculator();
        var service = new InvoiceService(db, calculator);

        // Act: ujemny VAT powinien przerwac caly proces
        Assert.Throws<ArgumentException>(
            () => service.CreateInvoice("Blad Sp. z o.o.", 100, -10));

        // Assert: baza ma pozostac pusta
        Assert.Empty(db.SavedInvoices);
    }

    [Fact]
    public void Integration_ShouldCorrectlyAccumulateMultipleInvoicesInDatabase()
    {
        // Arrange
        var db = new InvoiceDatabase();
        var calculator = new TaxCalculator();
        var service = new InvoiceService(db, calculator);

        // Act: dwie faktury w tej samej bazie
        service.CreateInvoice("Klient A", 100, 23);
        service.CreateInvoice("Klient B", 200, 0);

        // Assert
        Assert.Equal(2, db.SavedInvoices.Count);
        var first = db.SavedInvoices.FirstOrDefault(i => i.Customer == "Klient A");
        var second = db.SavedInvoices.FirstOrDefault(i => i.Customer == "Klient B");
        Assert.Equal(123m, first?.GrossPrice);
        Assert.Equal(200m, second?.GrossPrice);
    }
}
```

```
}
```

Uruchomienie testów

Testy uruchamiamy w katalogu rozwiązania poleceniem:

```
dotnet test
```

Po poprawnym wykonaniu wszystkie cztery testy powinny zostać zaliczone (Passed: 4, Failed: 0).

Wnioski do rozdziału

Testy integracyjne potwierdziły, że serwis, kalkulator i baza danych współpracują poprawnie: poprawne dane są przeliczane i zapisywane, a błędne dane przerywają proces bez pozostawiania śladu w bazie. Dzięki użyciu prawdziwych obiektów zamiast atrap test sprawdza realne zachowanie systemu, a nie tylko pojedynczą metodę.

3. Testy systemowe

Testy systemowe (ang. system tests) to poziom testowania, na którym sprawdzamy kompletny, zintegrowany system jako całość – tak, jak będzie go używał użytkownik. Test nie wywołuje pojedynczych metod ani nie zagląda do kodu: podaje dane przez interfejs aplikacji (w tej pracy jest nim wiersz poleceń) i porównuje to, co system zwrócił, z wymaganiami. Są to więc testy czarnoskrzynkowe.

Testy systemowe weryfikują, czy system spełnia założone wymagania funkcjonalne, a także czy sensownie reaguje na błędne dane i nieprawidłowe polecenia. W większych projektach obejmują one również m.in. wydajność, bezpieczeństwo i użyteczność. Nawet jeśli wszystkie klasy i ich połączenia zostały sprawdzone wcześniej, dopiero test całego systemu pokazuje, że użytkownik dostaje to, czego oczekuje.

Kiedy wykonujemy testy systemowe?

- po zakończeniu testów integracyjnych, gdy wszystkie moduły są już połączone w kompletną aplikację,
- w środowisku możliwie zbliżonym do produkcyjnego,
- przed testami akceptacyjnymi, czyli zanim system zobaczy klient,
- po większych zmianach w systemie, aby upewnić się, że nic się nie zepsuło (testy regresji).

Porównanie z testami integracyjnymi

Cecha	Testy integracyjne	Testy systemowe
Zakres	kilka współpracujących komponentów	cały system
Punkt wejścia	wywołanie metod serwisu	interfejs użytkownika (tu: polecenia tekstowe)
Wiedza o kodzie	zwykle potrzebna	zbędna (czarna skrzynka)
Co wykrywają	błędy na styku komponentów	błędy w działaniu systemu widoczne dla użytkownika

Przykład (opisowy)

W sklepie internetowym test systemowy odtwarza pracę prawdziwego klienta: wchodzi na stronę, wyszukuje produkt, dodaje go do koszyka, podaje dane do wysyłki, płaci i sprawdza, czy pojawiło się potwierdzenie zamówienia, a zamówienie jest widoczne w panelu sprzedawcy. Test nie interesuje się tym, które klasy wykonały pracę – liczy się wyłącznie wynik widoczny z zewnątrz.

Narzędzie: xUnit

xUnit nie ogranicza się do testów jednostkowych – o rodzaju testu decyduje jego zakres, a nie narzędzie. W testach systemowych używamy go do uruchomienia całej aplikacji i sprawdzenia jej wyjścia. Zamiast klawiatury i ekranu aplikacja dostaje strumienie z biblioteki `System.IO`: `StringReader` (podaje skrypt poleceń, jakby wpisywał je użytkownik) oraz `StringWriter` (zbiera wszystko, co program wypisał). Dzięki temu test przechodzi przez cały system tak samo, jak prawdziwa sesja w konsoli. Najważniejsze elementy:

- `[Fact]` – atrybut oznaczający metodę jako test,
- `Assert.Equal` – porównanie oczekiwanego i rzeczywistego wyniku (także całych tablic),
- `StringReader` i `StringWriter` – podstawienie wejścia i przechwycenie wyjścia programu,
- `dotnet test` – polecenie uruchamiające wszystkie testy w rozwiązaniu.

3.1 Implementacja programu

Jako program testowany rozbudowano system **SuperProstyFaktury** z rozdziału 2 o interfejs tekstowy. Aplikacja czyta polecenia ze standardowego wejścia i wypisuje wyniki na standardowe wyjście, a pod spodem korzysta z tych samych klas co wcześniej: `InvoiceService`, `TaxCalculator` i `InvoiceDatabase`. Dzięki temu test systemowy obejmuje cały system – od wpisania polecenia, przez przeliczenie ceny i zapis w bazie, aż po wypisanie wyniku.

Struktura rozwiązania

```
SuperProstyFaktury.sln
 +-- SuperProstyFaktury/        (biblioteka klas - kod produkcyjny)
 |     +-- Invoice.cs
 |     +-- TaxCalculator.cs
 |     +-- InvoiceDatabase.cs
 |     +-- InvoiceService.cs
 |     +-- ConsoleApp.cs        (NOWA - interfejs tekstowy)
 +-- SuperProstyFaktury.App/    (NOWY - aplikacja konsolowa)
 |     +-- Program.cs
 +-- SuperProstyFaktury.Tests/  (projekt xUnit - testy)
       +-- InvoiceIntegrationTests.cs
       +-- InvoiceSystemTests.cs  (NOWY)
```

Do rozwiązania dodano aplikację konsolową poleceniami:

```
dotnet new console -n SuperProstyFaktury.App
dotnet sln add SuperProstyFaktury.App
dotnet add SuperProstyFaktury.App reference SuperProstyFaktury
```

Opis działania

Użytkownik steruje programem krótkimi poleceniami wpisywanymi w kolejnych wierszach:

- `dodaj <klient> <netto> <vat>` – wystawia fakturę (nazwa klienta to jedno słowo, liczby z kropką),
- `lista` – wypisuje wszystkie zapisane faktury wraz z ceną brutto,
- `suma` – wypisuje łączną wartość brutto wszystkich faktur,
- `pomoc` – wypisuje listę poleceń,
- `koniec` – żegna użytkownika i kończy program.

Cała logika znajduje się w klasie `ConsoleApp`. Jej metoda `Run` przyjmuje `TextReader` i `TextWriter`, a nie wprost `Console.In` i `Console.Out`. W aplikacji przekazujemy do niej prawdziwą konsolę (plik `Program.cs`), a w testach `StringReader` i `StringWriter` – kod programu jest przy tym identyczny, więc test sprawdza dokładnie to, co zobaczyłby użytkownik.

Program reaguje też na błędy: gdy liczba argumentów jest zła lub cena albo VAT nie są liczbą, wypisuje `Blad: niepoprawne dane`; gdy VAT jest ujemny, wypisuje komunikat wyjątku z kalkulatora (baza pozostaje wtedy bez zmian); nieznane polecenie kończy się komunikatem `Nieznane polecenie`. Puste wiersze są ignorowane. Liczby są wczytywane i wypisywane w formacie niezależnym od ustawień regionalnych (kropka dziesiętna, dwa miejsca po przecinku), żeby wynik testów był taki sam na każdym komputerze.

Klasy `Invoice`, `TaxCalculator`, `InvoiceDatabase` i `InvoiceService` pozostały bez zmian (kod w rozdziale 2.1). Nowe pliki wyglądają następująco:

Kod programu

```
// ConsoleApp.cs
using System;
using System.Globalization;
using System.IO;

namespace SuperProstyFaktury
{
    public class ConsoleApp
    {
        private readonly InvoiceDatabase _db = new();
        private readonly InvoiceService _service;

        public ConsoleApp()
        {
            _service = new InvoiceService(_db, new TaxCalculator());
        }

        public void Run(TextReader input, TextWriter output)
        {
            output.WriteLine("SuperProstyFaktury - wpisz 'pomoc', aby zobaczyc polecenia.");

            string? line;
            while ((line = input.ReadLine()) != null)
            {
                var parts = line.Split(' ', StringSplitOptions.RemoveEmptyEntries);
                if (parts.Length == 0)
                    continue;

                switch (parts[0].ToLowerInvariant())
                {
                    case "dodaj":
                        Add(parts, output);
                        break;
                    case "lista":
                        List(output);
                        break;
                    case "suma":
                        output.WriteLine("Suma: " + Format(Sum()));
                        break;
                    case "pomoc":
                        output.WriteLine("Polecenia: dodaj <klient> <netto> <vat>, lista, suma, koniec");
                        break;
                    case "koniec":
                        output.WriteLine("Do widzenia!");
                        return;
                    default:
                        output.WriteLine("Nieznane polecenie: " + parts[0]);
                        break;
                }
            }
        }

        private void Add(string[] parts, TextWriter output)
        {
            if (parts.Length != 4
                || !decimal.TryParse(parts[2], NumberStyles.Number,
                                     CultureInfo.InvariantCulture, out var net)
                || !decimal.TryParse(parts[3], NumberStyles.Number,
                                     CultureInfo.InvariantCulture, out var vat))
            {
                output.WriteLine("Blad: niepoprawne dane");
                return;
            }

            try
            {
                _service.CreateInvoice(parts[1], net, vat);
                var saved = _db.SavedInvoices[_db.SavedInvoices.Count - 1];
                output.WriteLine("Dodano fakture: " + saved.Customer + " - " + Format(saved.GrossPrice));
```

```
            }
            catch (ArgumentException ex)
            {
                output.WriteLine("Blad: " + ex.Message);
            }
        }

        private void List(TextWriter output)
        {
            if (_db.SavedInvoices.Count == 0)
            {
                output.WriteLine("Brak faktur.");
                return;
            }

            for (int i = 0; i < _db.SavedInvoices.Count; i++)
            {
                var invoice = _db.SavedInvoices[i];
                output.WriteLine((i + 1) + ". " + invoice.Customer + " - " + Format(invoice.GrossPrice));
            }
        }

        private decimal Sum()
        {
            decimal sum = 0;
            foreach (var invoice in _db.SavedInvoices)
                sum += invoice.GrossPrice;
            return sum;
        }

        private static string Format(decimal value) =>
            value.ToString("0.00", CultureInfo.InvariantCulture);
    }
}
```

```
// Program.cs (projekt SuperProstyFaktury.App)
using System;
using SuperProstyFaktury;

new ConsoleApp().Run(Console.In, Console.Out);
```

3.2 Implementacja testów

Dla systemu napisano cztery testy systemowe w xUnit. Każdy test tworzy nową aplikację (`ConsoleApp`), podaje jej skrypt poleceń, tak jak zrobiłby to użytkownik, i porównuje cały tekst wypisany przez program z oczekiwanym wynikiem, wiersz po wierszu. Pomocnicza metoda `RunSession` przyjmuje listę poleceń, uruchamia aplikację na `StringReader` i `StringWriter` i zwraca wypisane wiersze (bez zależności od tego, czy system używa znaku końca wiersza `\n`, czy `\r\n`). Każdy test korzysta z własnej instancji aplikacji, więc testy są od siebie niezależne.

Opis testów

Test	Co sprawdza	Oczekiwany wynik
`System_ShouldHandleFullUserSession`	Pełna sesja użytkownika: dwie faktury (23% i 8% VAT), polecenia lista i suma, wyjście z programu.	komunikaty o dodaniu, lista z cenami 1230.00 i 540.00, suma 1770.00, pożegnanie
`System_ShouldReportNegativeVatAndKeepWorking`	Ujemny VAT jest zgłoszony, nic nie trafia do bazy, a system nadal działa i przyjmuje poprawną fakturę.	komunikat błędu, „Brak faktur.”, po poprawnej fakturze lista z jedną pozycją 123.00
`System_ShouldRespondToInvalidInputAndUnknownCommands`	Błędne dane, zbyt mało argumentów, nieznane polecenie i pusty wiersz.	dwa razy „Blad: niepoprawne dane”, „Nieznane polecenie: latanie”, pusty wiersz pominięty
`System_ShouldStopProcessingAfterExitCommand`	Polecenia wpisane po koniec nie mogą zostać wykonane.	tylko powitanie i „Do widzenia!”

Kod testów

Plik `InvoiceSystemTests.cs` w projekcie `SuperProstyFaktury.Tests`:

```
using System;
using System.IO;
using Xunit;
using SuperProstyFaktury;

namespace SuperProstyFaktury.Tests
{
    public class InvoiceSystemTests
    {
        // Uruchamia cala aplikacje tak, jak robi to uzytkownik w konsoli:
        // polecenia wchodza przez StringReader, wynik zbiera StringWriter.
        private static string[] RunSession(params string[] commands)
        {
            var input = new StringReader(string.Join("\n", commands));
            var output = new StringWriter();

            new ConsoleApp().Run(input, output);

            return output.ToString()
                .Split(new[] { "\r\n", "\n" }, StringSplitOptions.RemoveEmptyEntries);
        }

        [Fact]
        public void System_ShouldHandleFullUserSession()
        {
            var result = RunSession(
                "dodaj Firma 1000 23",
                "dodaj Ksiegarnia 500 8",
                "lista",
                "suma",
                "koniec");

            var expected = new[]
            {
                "SuperProstyFaktury - wpisz 'pomoc', aby zobaczyc polecenia.",
                "Dodano fakture: Firma - 1230.00",
                "Dodano fakture: Ksiegarnia - 540.00",
                "1. Firma - 1230.00",
                "2. Ksiegarnia - 540.00",
```

```
                "Suma: 1770.00",
                "Do widzenia!"
            };
            Assert.Equal(expected, result);
        }

        [Fact]
        public void System_ShouldReportNegativeVatAndKeepWorking()
        {
            var result = RunSession(
                "dodaj Blad 100 -10",
                "lista",
                "dodaj Klient 100 23",
                "lista",
                "koniec");

            var expected = new[]
            {
                "SuperProstyFaktury - wpisz 'pomoc', aby zobaczyc polecenia.",
                "Blad: VAT nie moze byc ujemny",
                "Brak faktur.",
                "Dodano fakture: Klient - 123.00",
                "1. Klient - 123.00",
                "Do widzenia!"
            };
            Assert.Equal(expected, result);
        }

        [Fact]
        public void System_ShouldRespondToInvalidInputAndUnknownCommands()
        {
            var result = RunSession(
                "dodaj Firma abc 23",
                "dodaj Firma",
                "latanie",
                "",
                "koniec");

            var expected = new[]
            {
                "SuperProstyFaktury - wpisz 'pomoc', aby zobaczyc polecenia.",
                "Blad: niepoprawne dane",
                "Blad: niepoprawne dane",
                "Nieznane polecenie: latanie",
                "Do widzenia!"
            };
            Assert.Equal(expected, result);
        }

        [Fact]
        public void System_ShouldStopProcessingAfterExitCommand()
        {
            var result = RunSession(
                "koniec",
                "dodaj Firma 100 23",
                "lista");

            var expected = new[]
            {
                "SuperProstyFaktury - wpisz 'pomoc', aby zobaczyc polecenia.",
                "Do widzenia!"
            };
            Assert.Equal(expected, result);
        }
    }
}
```

Uruchomienie testów

Testy uruchamiamy w katalogu rozwiązania poleceniem:

```
dotnet test
```

Samą aplikację można też uruchomić ręcznie poleceniem `dotnet run --project SuperProstyFaktury.App` i wpisywać polecenia z klawiatury. Po poprawnym wykonaniu wszystkie testy w rozwiązaniu (4 integracyjne i 4 systemowe) powinny zostać zaliczone (Passed: 8, Failed: 0). Same testy systemowe można uruchomić poleceniem `dotnet test --filter InvoiceSystemTests` (Passed: 4, Failed: 0).

Wnioski do rozdziału

Testy systemowe potwierdziły, że aplikacja jako całość działa zgodnie z oczekiwaniami użytkownika: poprawne polecenia są przeliczane, zapisywane i wypisywane w czytelnej postaci, a błędne dane i nieznane polecenia kończą się zrozumiałym komunikatem, bez uszkodzenia stanu systemu. Test nie wywoływał żadnej metody serwisu ani kalkulatora – sprawdzał wyłącznie wejście i wyjście programu, czyli to, co widzi użytkownik, co jest istotą testowania czarnoskrzynkowego na poziomie systemu.